

ZYASTRA

AI VOICE API

Enterprise Integration Guide

Prepared for Client Integration Teams

Document Version 1.0 | July 2026

Confidential — For Authorized Recipients Only

1. Overview

The Zyastra AI Voice API enables programmatic triggering of AI-driven voice agents for candidate pre-screening, lead qualification, and structured data collection. Integrators can initiate real-time phone conversations, receive live transcripts and recordings, and extract structured variables from each call via signed webhooks.

This guide covers authentication, the core API endpoints, webhook verification, rate limits, and error handling required for a production integration.

1.1 Integration Flow

A typical integration follows three steps:

- Authenticate requests using your issued API key and secret.
- Trigger a call via HTTP POST to `/voice/trigger`, supplying candidate details and an AI agent prompt.
- Receive results via an HMAC-signed HTTPS webhook once the call completes, containing the transcript, recording, and extracted variables.

Step	Action	Direction
1	Your platform sends a POST request to <code>/voice/trigger</code>	Your Platform → Zyastra
2	Zyastra places a real-time AI phone call to the candidate	Zyastra → Candidate
3	Zyastra sends a signed webhook event with results	Zyastra → Your Platform

2. Authentication & Base URL

2.1 Production Base URL

```
https://astraapi.zyvka.com/api/v1/external
```

2.2 Authentication Headers

All API requests must include the following headers:

```
Content-Type: application/json
x-api-key: YOUR_API_KEY
x-api-secret: YOUR_API_SECRET
```

Security Note: API secrets must be kept confidential and stored in environment variables. All requests should be executed from your server-side backend, never from client-side code.

3. API Reference

3.1 Trigger AI Voice Call

Initiates an AI voice call to a candidate or lead using dynamic parameters.

Method: POST **Endpoint:** `/voice/trigger`

Request Payload Schema

```
{
  "candidate": {
    "phoneNumber": "+919876543210",
    "firstName": "Rahul",
    "lastName": "Sharma",
    "email": "rahul.sharma@example.in"
  },
  "agent": {
    "prompt": "You are Ananya, a senior HR recruiter from TechCorp Bangalore. Speak with Rahul regarding his Node.js and React experience. Confirm his current CTC, expected CTC in LPA, notice period, and willingness to work in hybrid mode.",
    "firstMessage": "Namaste Rahul, this is Ananya from TechCorp HR team. Am I speaking with Rahul?",
    "preferredLanguage": "en-IN"
  },
  "voiceConfiguration": {
    "engine": "regional-std",
    "speed": 1.0
  },
  "webhook": {
    "url": "https://your-company.com/webhooks/voice-results",
    "secret": "whsec_your_custom_secret_key_12345"
  },
  "analysisVariables": [
    "candidate_interest_score",
    "notice_period_days",
    "current_ctc_lpa",
    "expected_ctc_lpa",
    "relocation_willingness"
  ],
  "metadata": {
    "clientJobId": "JOB-BLR-2026-08",
    "applicantId": "APP-99120"
  },
  "dynamicParams": {
    "maxDurationSeconds": 600,
    "silenceTimeoutSeconds": 15
  }
}
```

Field Specifications

Parameter	Type	Required	Description
candidate.phoneNumber	String	Yes	Phone number in E.164 format (e.g. +91987654XXX0, +1415555XXX1)
candidate.firstName	String	Yes	Candidate's first name

Parameter	Type	Required	Description
candidate.lastName	String	No	Candidate's last name
candidate.email	String	No	Candidate's email address
agent.customAgentId	String	Conditional*	ID of a pre-configured agent (*required if prompt is omitted)
agent.prompt	String	Conditional*	Dynamic instructions for the AI recruiter (*required if customAgentId is omitted)
agent.firstMessage	String	No	Opening line spoken by the AI agent upon call pick-up
agent.preferredLanguage	String	No	Spoken language code (e.g. en-IN, hi-IN, ta-IN, te-IN, en-US). Default: en-IN
voiceConfiguration.engine	String	No	"regional-std" (India) or "global-std" (International). Defaults automatically based on phone prefix
webhook.url	String	No	HTTPS endpoint to receive webhook call completion events
webhook.secret	String	No	Secret key used to compute the HMAC-SHA256 signature
analysisVariables	Array	No	Keys to extract from the call conversation during post-call analysis
metadata	Object	No	Key-value dictionary echoed back in the webhook payload for tracking

Success Response — HTTP 200 OK

```
{
  "success": true,
  "message": "Voice call initiated successfully",
  "data": {
    "requestId": "req_1722441600_a8f9b2",
    "callId": "clx_9981247a",
    "callReferenceId": "call_ref_8829104",
    "status": "queued",
    "candidate": {
      "phoneNumber": "+919876543210",
      "firstName": "Rahul"
    },
    "engine": "regional-std",
    "createdAt": "2026-07-31T21:06:41.000Z"
  },
  "metadata": {
    "apiVersion": "v2",
    "timestamp": "2026-07-31T21:06:41.115Z",
    "processingTimeMs": 115
  }
}
```

3.2 Get Call Details & Transcript

Retrieves real-time status, transcript, and audio recording link for a call.

Method: GET **Endpoint:** `/voice/call/:callId`

Response — HTTP 200 OK

```
{
  "success": true,
  "data": {
    "callId": "clx_9981247a",
    "callReferenceId": "call_ref_8829104",
    "status": "completed",
    "candidate": {
      "phoneNumber": "+919876543210",
      "name": "Rahul Sharma"
    },
    "durationSeconds": 185,
    "transcript": "AI: Namaste Rahul... \nCandidate: Yes, speaking...",
    "recordingUrl": "https://storage.zyastra.com/recordings/clx_9981247a.wav",
    "createdAt": "2026-07-31T21:06:41.000Z",
    "updatedAt": "2026-07-31T21:09:46.000Z"
  }
}
```

4. Webhook Integration & Verification

Upon call completion or failure, Zyastra sends an HTTP POST notification to the configured webhook.url.

4.1 Webhook Headers

Header	Value
Content-Type	application/json
X-Zyastra-Signature	t=<timestamp>,v1=<hmac_sha256_signature>
X-Zyastra-Event	call.completed or call.failed

4.2 Signature Verification

Verify the authenticity of each webhook by computing an HMAC-SHA256 signature over the string <timestamp>.<raw_json_body> using your webhook secret, then comparing it to the signature supplied in the X-Zyastra-Signature header.

Node.js / TypeScript

```
import crypto from 'crypto';

export function verifyZyastraWebhook(
  rawBody: string,
  signatureHeader: string,
  secret: string
): boolean {
  const parts = signatureHeader.split(',');
  const tPart = parts.find(p => p.startsWith('t='));
  const v1Part = parts.find(p => p.startsWith('v1='));
}
```

```

if (!tPart || !v1Part) return false;

const timestamp = tPart.split('=')[1];
const signature = v1Part.split('=')[1];

const expectedSignature = crypto
  .createHmac('sha256', secret)
  .update(`${timestamp}.${rawBody}`)
  .digest('hex');

return crypto.timingSafeEqual(
  Buffer.from(signature, 'hex'),
  Buffer.from(expectedSignature, 'hex')
);
}

```

Python

```

import hmac
import hashlib

def verify_zyastra_webhook(raw_body: str, signature_header: str,
secret: str) -> bool:
    parts = dict(part.split('=') for part in
signature_header.split(','))
    timestamp = parts.get('t')
    signature = parts.get('v1')

    if not timestamp or not signature:
        return False

    computed = hmac.new(
        secret.encode('utf-8'),
        f"{timestamp}.{raw_body}".encode('utf-8'),
        hashlib.sha256
    ).hexdigest()

    return hmac.compare_digest(signature, computed)

```

4.3 Webhook Payload Schema

```

{
  "event": "call.completed",
  "eventId": "evt_1722441600_9f81a2",
  "timestamp": "2026-07-31T21:10:00.000Z",
  "data": {
    "callId": "clx_9981247a",
    "callReferenceId": "call_ref_8829104",
    "status": "completed",
    "durationSeconds": 185,
    "timestamps": {
      "startTime": "2026-07-31T21:07:00.000Z",
      "endTime": "2026-07-31T21:09:45.000Z"
    },
    "candidate": {

```

```

    "phoneNumber": "+919876543210",
    "firstName": "Rahul",
    "lastName": "Sharma"
  },
  "transcript": "AI: Namaste Rahul... \nCandidate: Yes,
speaking...",
  "recordingUrl":
"https://storage.zyastra.com/recordings/clx_9981247a.wav",
  "variables": {
    "candidate_interest_score": 9.0,
    "notice_period_days": 15,
    "current_ctc_lpa": "14 LPA",
    "expected_ctc_lpa": "18 LPA",
    "relocation_willingness": "Yes (Bangalore)"
  },
  "summary": "Candidate demonstrated strong technical
proficiency,
15 days notice period, expected CTC 18 LPA.",
  "metadata": {
    "clientJobId": "JOB-BLR-2026-08",
    "applicantId": "APP-99120"
  }
}

```

5. Rate Limits & Error Codes

5.1 Rate Limits

Each API account is provisioned with an hourly sliding-window limit (default: 100 requests/hour). Every response includes the current quota status:

- X-RateLimit-Limit — maximum requests permitted per hour
- X-RateLimit-Remaining — remaining request quota for the current window
- X-RateLimit-Reset — UTC ISO timestamp when the quota resets

5.2 Error Codes Reference

HTTP Status	Error Code	Description	Troubleshooting
400	INVALID_PAYLOAD	Missing required fields	Check candidate.phoneNumber and candidate.firstName
400	INVALID_PHONE_NUMBER	Phone number formatting error	Use strict E.164 format (e.g. +919876543210)
401	UNAUTHORIZED_API_KEY	Invalid API key or secret	Verify x-api-key and x-api-secret headers
404	AGENT_NOT_FOUND	customAgentId does not exist	Verify the custom agent ID in the dashboard
429	RATE_LIMIT_EXCEEDED	Request limit exceeded	Space out requests or contact support for a limit increase
503	ENGINE_UNAVAILABLE	Voice service temporarily busy	Retry after a backoff delay

6. Code Example

6.1 Node.js / Axios

```
const axios = require('axios');

async function triggerCall() {
  try {
    const res = await axios.post(
      'https://astraapi.zyvka.com/api/v1/external/voice/trigger',
      {
        candidate: {
          phoneNumber: '+919876543210',
          firstName: 'Rahul',
          lastName: 'Sharma'
        },
        agent: {
          prompt: 'You are Ananya, an HR recruiter. Screen Rahul for the Node.js developer role.',
          preferredLanguage: 'en-IN'
        },
        webhook: {
          url: 'https://your-company.com/webhooks/voice-results',
          secret: 'whsec_secret123'
        },
        analysisVariables: [
          'candidate_interest_score',
          'notice_period_days',
          'expected_ctc_lpa'
        ],
        metadata: { clientJobId: 'JOB-BLR-2026-08' }
      },
      {
        headers: {
          'Content-Type': 'application/json',
          'x-api-key': 'YOUR_API_KEY',
          'x-api-secret': 'YOUR_API_SECRET'
        }
      }
    );

    console.log('Call Triggered Successfully:', res.data);
  } catch (err) {
    console.error('Error:', err.response ? err.response.data : err.message);
  }
}

triggerCall();
```

7. Support & Assistance

For technical support, custom rate limits, or integration questions, please contact your Zyastra Solutions Engineering Team. **Email:** support@zyvka.com